

SEO PACKAGE

1. SEO Title

SQL Aggregate Functions Explained: COUNT, SUM, AVG, MIN, and MAX for Beginners

2. SEO Subtitle

Learn how the five core SQL aggregate functions collapse entire tables into single, meaningful answers — and how to avoid the NULL traps that quietly break your results.

3. Featured Quote

"Aggregate functions share one habit: many rows go in, one answer comes out — and every one of them quietly skips NULL, except `COUNT(*)`."

4. Meta Description

Master SQL COUNT, SUM, AVG, MIN, and MAX. Learn when to use each, why NULL isn't zero, and the difference between COUNT(*) and COUNT(column). (154 chars)

5. URL Slug

`sql-aggregate-functions-count-sum-avg-min-max`

6. Keywords

SQL aggregate functions, COUNT in SQL, SUM in SQL, AVG in SQL, MIN and MAX SQL, COUNT(*) vs COUNT(column), SQL NULL handling, COALESCE SQL, SQL for beginners, learn SQL, SQL tutorial, database aggregation, SQL data analysis, GROUP BY SQL, SQL interview questions, SQL functions explained, aggregate query SQL, SQL summary statistics, SQL null vs zero, relational database basics

7. Tags

SQL, Database, Aggregate Functions, COUNT, SUM, AVG, MIN, MAX, Data Analysis, SQL Tutorial, Learn SQL, NULL Handling, Coding Interview, Backend Development, Data Engineering

SQL Aggregate Functions: How COUNT, SUM, AVG, MIN, and MAX Turn Whole Tables Into a Single Answer

8. Introduction

Picture a shoebox stuffed with a year of receipts. You do not want to read each one. You want one number: *how much did I spend?* That single question — collapsing a mountain of rows into one meaningful value — is the entire job of **SQL aggregate functions**.

If you are learning SQL, or preparing for a **coding interview** that touches databases, this is one of the most practical topics you will ever study. Almost every dashboard, analytics report, billing summary, and "total users" badge you have ever seen is powered by `COUNT`, `SUM`, `AVG`, `MIN`, or `MAX`. These five functions are the difference between shipping a million rows across the network to your application and asking the database for one tidy answer.

Interviewers love this area because it quietly tests whether you actually understand your data. Do you know that `NULL` is not the same as zero? Can you explain why `COUNT(*)` and `COUNT(phone)` return different numbers? Do you know why mixing a raw column with an aggregate makes the database refuse to run your query? These are not trivia questions — they are the exact places where real production reports go silently wrong.

By the end of this article, you will reach for the right function without thinking, and — just as importantly — you will know the traps that catch nearly every beginner.

9. Problem Overview

An **aggregate function** takes a whole column of values and returns a single value. The word *aggregate* simply means "gather into one." Many rows go in; one answer comes out.

That is the mental model to lock in. Every function we cover follows the same shape — it scans a column and funnels every value down into one result. What differs is the *math* applied on the way down:

Function	Question it answers	Works on
COUNT	How many?	Any type
SUM	How much in total?	Numbers only
AVG	What's the typical value?	Numbers only
MIN	What's the smallest / earliest?	Numbers, dates, text
MAX	What's the largest / latest?	Numbers, dates, text

There is one shared behavior that will save you hours of debugging: **every aggregate function ignores NULL values**, with a single famous exception — `COUNT(*)`, which counts every row no matter what.

10. Example

Let's build a small `orders` table to work with throughout the article.

```
CREATE TABLE orders (
  id      INTEGER,
  customer TEXT,
  amount  INTEGER,
  phone   TEXT,
  ordered_on DATE
);

INSERT INTO orders VALUES
  (1, 'Ava', 10, '555-0100', '2026-01-05'),
  (2, 'Ben', 20, NULL, '2026-02-14'),
  (3, 'Ava', 30, '555-0199', '2026-03-01');
```

Now run each function:

```
SELECT
  COUNT(*)      AS total_rows,      -- 3
  COUNT(phone)  AS phones_on_file,  -- 2 (Ben's is NULL)
  SUM(amount)   AS revenue,         -- 60
  AVG(amount)   AS typical_order,  -- 20
  MIN(amount)   AS smallest,        -- 10
  MAX(amount)   AS largest,         -- 30
  MIN(ordered_on) AS first_order,   -- 2026-01-05
  MAX(ordered_on) AS last_order     -- 2026-03-01
FROM orders;
```

Walk through the outputs:

- **total_rows = 3** — there are three rows, full stop. **COUNT(*)** never skips anything.
- **phones_on_file = 2** — Ben left his phone blank, so **COUNT(phone)** ignores that row. This is the single most important distinction in the whole lesson.
- **revenue = 60** — $10 + 20 + 30$.
- **typical_order = 20** — $60 / 3$.
- **smallest / largest** — the extremes of the **amount** column.
- **first_order / last_order** — notice **MIN** and **MAX** work beautifully on **dates**. The earliest date is the "smallest," the latest is the "largest." This is an underrated way to find a customer's first or last order.

11. Intuition

Here is how an experienced engineer thinks about this, before writing a single line of SQL.

Whenever you have a column and want to describe it with **one number**, you are aggregating. So the first question is never "which function?" — it is *"what single thing do I want to know about this column?"*

Say the sentence out loud:

- "How **many**?" → **COUNT**
- "How **much** altogether?" → **SUM**
- "What's the **typical** value?" → **AVG**
- "What's the **smallest / largest**?" → **MIN / MAX**

COUNT and **SUM** feel similar to beginners, but they answer opposite questions. *How many orders?* is three. *How much money?* is sixty. When you're stuck, the plain-English sentence picks the function for you.

The second instinct an experienced engineer has is suspicion about **empty cells**. Real data is messy — phone numbers are blank, ratings are missing, timestamps never got filled in. So before trusting any aggregate, they ask: *what does this function do with the blanks?* The answer, for four of the five, is "pretend they don't exist." That single fact prevents the most common category of quietly-wrong reports.

12. Brute Force Solution

The idea: don't use SQL aggregation at all. Pull every row into your application and compute the summary in Python (or whatever language you use).

```
import sqlite3

conn = sqlite3.connect("shop.db")
rows = conn.execute("SELECT amount FROM orders").fetchall()

amounts = [r[0] for r in rows if r[0] is not None] # skip NULLs by hand
total    = sum(amounts)
count    = len(amounts)
average  = total / count if count else 0
```

Algorithm: fetch all rows → filter out `None` → loop and compute totals in application code.

Advantages: - Full flexibility — you have a real programming language, so any weird custom logic is possible. - Easy to unit test with familiar tools.

Disadvantages: - You ship potentially **millions of rows across the network** just to produce one small number. - Your application does work the database is optimized to do faster. - You have to remember to handle `NULL` yourself — forget the `is not None` filter and your average is wrong.

Complexity: Time is $O(n)$ in your app *plus* the network transfer of all n rows. Space is $O(n)$ because you materialize every row in memory.

This is fine for a handful of rows. It becomes painful the moment your table grows.

13. Optimal Solution

Let the database do the counting. It sits right next to the data, computes the answer in a single pass, and returns **one tiny value** instead of the entire table.

```
SELECT AVG(amount) FROM orders;
```

The engine scans the `amount` column once, sums the non-NULL values, divides by how many there were, and hands back a single row. No network flood, no manual loop, no forgotten `NULL` filter.

Here is the "collapse" visualized:

```
amount column      aggregate
  10  |
  20  |—— SUM → 60
  30  |
```

```

10  |
20  |— AVG → 20 (NULLs are skipped, not counted as 0)
30  |
(null)

```

Notice the last diagram: even with a `NULL` in the mix, `AVG` divides by **3**, not 4. The blank row is invisible to it.

When SQL wins: almost always for summarizing data. It's faster, closer to the data, and less code.

When application code wins: genuinely custom logic that SQL's math can't express cleanly. That's the honest trade-off — SQL math is less flexible than a full programming language.

14. Python Code

Here is a clean, production-style helper that runs each aggregate and prints a small report. It uses Python's built-in `sqlite3`, so it runs anywhere with no dependencies.

```

import sqlite3

def build_demo_db() -> sqlite3.Connection:
    """Create an in-memory orders table with one intentional NULL phone."""
    conn = sqlite3.connect(":memory:")
    conn.execute(
        """
        CREATE TABLE orders (
            id          INTEGER PRIMARY KEY,
            customer    TEXT,
            amount      INTEGER,
            phone       TEXT,
            ordered_on  TEXT
        )
        """
    )
    conn.executemany(
        "INSERT INTO orders VALUES (?, ?, ?, ?, ?)",
        [
            (1, "Ava", 10, "555-0100", "2026-01-05"),
            (2, "Ben", 20, None, "2026-02-14"), # phone is NULL
            (3, "Ava", 30, "555-0199", "2026-03-01"),
        ],
    )
    return conn

def summarize(conn: sqlite3.Connection) -> dict:

```

```

"""Return the five core aggregates in a single database round trip."""
row = conn.execute(
    """
    SELECT
        COUNT(*)           AS total_rows,
        COUNT(phone)       AS phones_on_file,
        SUM(amount)        AS revenue,
        AVG(amount)        AS typical_order,
        MIN(amount)        AS smallest,
        MAX(amount)        AS largest,
        MIN(ordered_on)    AS first_order,
        MAX(ordered_on)    AS last_order,
        -- Treat missing amounts as zero on purpose using COALESCE:
        AVG(COALESCE(amount, 0)) AS avg_with_blanks_as_zero
    FROM orders
    """
).fetchone()

keys = [
    "total_rows", "phones_on_file", "revenue", "typical_order",
    "smallest", "largest", "first_order", "last_order",
    "avg_with_blanks_as_zero",
]
return dict(zip(keys, row))

if __name__ == "__main__":
    conn = build_demo_db()
    report = summarize(conn)

    # Self-check: the numbers the article promised.
    assert report["total_rows"] == 3
    assert report["phones_on_file"] == 2      # NULL phone is skipped
    assert report["revenue"] == 60
    assert report["typical_order"] == 20     # NULL never enters AVG
    assert report["first_order"] == "2026-01-05"
    assert report["last_order"] == "2026-03-01"

    for name, value in report.items():
        print(f"{name:24} = {value}")

```

15. Code Walkthrough

- **build_demo_db** creates an in-memory SQLite database, so the example is fully reproducible with zero setup. Ben's `phone` is deliberately `None` — that's what becomes SQL `NULL`.
- **summarize** runs all five functions in **one** query. This matters: one round trip to the database is far cheaper than nine separate ones.

- **COUNT(*)** vs **COUNT(phone)** sit side by side so the difference is impossible to miss — 3 versus 2.
- **AVG(amount)** returns 20, proving the **NULL** row is skipped, not treated as zero.
- **AVG(COALESCE(amount, 0))** shows the fix for when you *do* want blanks counted as zero — **COALESCE** swaps every **NULL** for 0 *before* the average runs.
- **The assert block** is a runnable self-check. If any aggregate ever behaves differently, the script fails loudly instead of printing wrong numbers.

16. Dry Run

Let's trace **AVG(amount)** by hand on our three rows plus an imagined blank.

Suppose the column holds: 10, 20, 30, **NULL**.

1. The engine scans the column.
2. It **discards the NULL** — it is invisible to **AVG**.
3. Remaining values: 10, 20, 30.
4. Sum = $10 + 20 + 30 = 60$.
5. Count of *non-null* values = 3.
6. Average = $60 / 3 = 20$.

The intuitive-but-wrong answer is $60 / 4 = 15$, treating the blank as a zero. SQL does **not** do that.

If you genuinely want 15, you must convert the blank first: **AVG(COALESCE(amount, 0))** gives $(10 + 20 + 30 + 0) / 4 = 15$.

17. Complexity Analysis

For a single aggregate over a table of **n** rows:

- **Time: O(n).** The engine makes one linear pass over the column, applying its running operation (add for **SUM**, compare for **MIN** / **MAX**, tally for **COUNT**). There is no way to summarize every value without at least looking at every value once — unless an index lets **MIN** / **MAX** jump straight to an end, which some databases optimize.
- **Space: O(1).** Each function keeps a tiny amount of running state — a running total, a running count, a current min. It does **not** hold all **n** rows in memory. That's precisely why aggregating in SQL beats pulling everything into your app, which costs O(n) space and O(n) network transfer.

18. Alternative Solutions

Aggregating per group with GROUP BY . So far every query produced *one* row for the whole table.

But what if you want a total *per customer*? You can't select `customer` alongside `SUM(amount)` without telling the database how to group — that's the next lesson:

```
SELECT customer, SUM(amount) AS spent
FROM orders
GROUP BY customer;
```

This returns one row per customer instead of one row overall. `GROUP BY` is the natural extension of everything here.

COUNT(DISTINCT col) . A close cousin of plain `COUNT` , it counts unique values only:

```
SELECT COUNT(DISTINCT customer) FROM orders; -- 2 (Ava, Ben), not 3
```

Compared to application-side aggregation (our brute force), SQL aggregation wins on performance and simplicity in nearly all reporting scenarios; app code only wins when the logic outgrows SQL's math.

19. Edge Cases

- **Empty table.** `COUNT(*)` returns `0` . But `SUM` , `AVG` , `MIN` , and `MAX` return `NULL` , not zero — there is nothing to compute. Guard for this: `COALESCE(SUM(amount), 0)` .
- **All values NULL.** `COUNT(col)` returns `0` ; `SUM` / `AVG` / `MIN` / `MAX` return `NULL` .
- **Duplicates.** Plain `COUNT` counts them; use `COUNT(DISTINCT col)` if you want uniques.
- **Single row.** Every function still works: `MIN` and `MAX` both return that one value.
- **Mixed / extreme values.** `MIN` and `MAX` correctly handle negatives, very large numbers, dates, and even text (lexicographic order for strings).

20. Common Mistakes

1. **Assuming NULL equals zero.** `AVG` skips blanks entirely. `10, 20, 30, NULL` averages to `20` , not `15` . Use `COALESCE(col, 0)` when you truly want blanks counted.
2. **Confusing COUNT(*) with COUNT(column)** . `COUNT(*)` counts rows; `COUNT(column)` counts non-NULL values. They answer different questions — neither is "wrong."
3. **Mixing a raw column with an aggregate.** `SELECT customer, SUM(amount) FROM orders;` fails (or returns nonsense) because one total can't sit beside fifty names. You need `GROUP BY` .

4. **Feeding text to SUM or AVG** . These are numeric-only. Running them on a text column is an error or a silent zero, depending on the database.
5. **Forgetting the empty-table NULL** . SUM over zero rows is NULL , not 0 . Reports that display it raw will show a blank instead of a friendly 0 .

21. Interview Questions

- What is the difference between COUNT(*) , COUNT(column) , and COUNT(DISTINCT column) ?
- If a column has NULLs, what does AVG return, and how would you make it treat NULLs as zero?
- Why does SELECT name, SUM(amount) FROM orders; fail? How do you fix it?
- What does SUM return over an empty table, and why?
- When would you aggregate in SQL versus in application code?
- How can you find a customer's first and last order date using aggregate functions?

22. Similar Problems

- [LeetCode 620 — Not Boring Movies](#) — filtering plus ordering, a natural companion to summarizing.
- [LeetCode 1741 — Find Total Time Spent by Each Employee](#) — SUM combined with GROUP BY , the exact next step after this lesson.
- [LeetCode 1050 — Actors and Directors Who Cooperated At Least Three Times](#) — COUNT with HAVING , aggregation used as a filter.
- [LeetCode 1075 — Project Employees I](#) — AVG with grouping and rounding.

They're related because each one moves aggregation from "whole table" to "per group," which is where this topic goes next.

23. Key Takeaways

- Aggregate functions collapse a column into a single answer: **many in, one out**.
- COUNT = how many, SUM = how much, AVG = the typical value, MIN / MAX = the extremes.
- **Every function ignores NULL** except COUNT(*) .
- NULL is **not** zero — use COALESCE when you need it to be.
- Don't mix raw columns with aggregates without GROUP BY .
- Aggregate in SQL by default; it's faster and closer to the data.

24. Watch the Video

Seeing the values *funnel down* into a single result makes this click far faster than reading about it.

Watch the full walkthrough — including the live `NULL` -versus-zero demo that trips up almost everyone — here: **{LINK}**

25. About the Series

This is **Lesson 7** of *Fun with Learning Technology* — a hands-on SQL series that teaches databases the way a senior engineer actually thinks about them. No memorization, no jargon dumps: just clear intuition, real examples, and the little traps that separate working queries from quietly-broken ones. Next up: **GROUP BY**, where these five functions go from summarizing the whole table to summarizing *each group* — one total per customer, per day, per anything.

26. Call To Action

If this made SQL aggregation click, here's how to help the series grow:

- 👍 **Like** the video on YouTube — it genuinely helps more developers find it.
- 🔔 **Subscribe** so you catch the `GROUP BY` lesson the moment it drops.
- 📌 **Subscribe on Substack** for the written deep-dives that go beyond each video.
- 💬 **Comment** with the aggregate-function bug that bit *you* — we've all had one.
- 📧 **Share** this with someone learning SQL or prepping for a coding interview.

SOCIAL MEDIA

LinkedIn Post

Most developers learn SQL's aggregate functions in about thirty seconds — and then spend months quietly shipping wrong numbers.

The functions themselves are simple. COUNT tells you how many. SUM tells you how much. AVG gives the typical value. MIN and MAX give the extremes. Every one of them takes a whole column and hands back a single answer.

The part that catches people isn't the syntax — it's NULL.

Here's the trap I fell into early: I had a column with some blanks, and I assumed AVG would treat them as zero. It doesn't. It pretends those rows don't exist. So 10, 20, 30, and a blank averages to 20 — not 15. If you actually want blanks counted as zero, you convert them first with COALESCE.

Two more distinctions worth internalizing:

- `COUNT()` counts every row. `COUNT(column)` skips the NULLs. If 100 customers exist but 20 left their phone blank, `COUNT()` says 100 and `COUNT(phone)` says 80. Neither is wrong — they answer different questions.
- You can't put a raw column next to an aggregate without `GROUP BY`. One grand total can't sit beside fifty customer names.

And the strategic call: aggregate in the database, not in your app. The database sits right next to the data and returns one tiny number instead of shipping a million rows across the network. Application code only wins when your logic outgrows SQL's math.

Master these five and you can summarize almost any table with confidence.

Full breakdown — with examples, complexity analysis, and the common mistakes — in the article. [Link in comments.](#)

SQL #Database #DataAnalysis

#SoftwareEngineering #LearnSQL

Twitter/X Thread

1/ SQL aggregate functions look trivial. COUNT, SUM, AVG, MIN, MAX.

But they hide one behavior that quietly corrupts real reports. Here's the whole toolkit in 10 tweets 🧵

2/ The core idea: many rows go in, ONE answer comes out.

An aggregate takes a whole column and collapses it to a single value. That's it. They just differ in the math on the way down.

3/ COUNT = "how many?" It's the headcount at the door.

`COUNT(*)` counts every row, no exceptions.

4/ But `COUNT(phone)` quietly SKIPS the blanks (NULLs).

100 customers exist → `COUNT(*) = 100`. 20 left phone blank → `COUNT(phone) = 80`.

Neither is wrong. They answer different questions.

5/ SUM = "how much altogether?" The cash register tape. $10 + 20 + 30 \rightarrow 60$.

Numbers only. Feed it text and it breaks.

6/ AVG = "what's the typical value?" Adds the column, divides by how many.

Here's the one that bit me: it **SKIPS** nulls. It does **NOT** treat a blank as zero.

10, 20, 30, blank → averages to 20. Not 15.

7/ Want blanks counted as zero? Convert them first:

`AVG(COALESCE(amount, 0))`

Now $(10+20+30+0)/4 = 15$.

8/ MIN and **MAX** = smallest and largest.

Underrated trick: they work on **DATES** too. `MIN(date)` = earliest order. `MAX(date)` = latest order.

Instant first/last lookup.

9/ Can't mix a raw column with an aggregate:

`SELECT name, SUM(amount) ...` ❌

One total can't sit next to 50 names. You need **GROUP BY** (that's the next level).

10/ The toolkit: • **COUNT** → how many • **SUM** → total • **AVG** → typical • **MIN/MAX** → extremes

All ignore **NULL** except `COUNT(*)`.

Say "how many vs how much" out loud and the right function picks itself. Full article + video 📌

Facebook Post

Ever stared at a table of numbers and just wanted **ONE** answer — the total, the average, the biggest? That's exactly what SQL's aggregate functions do. 🧮

In Lesson 7 we break down the five power tools: **COUNT** (how many), **SUM** (how much), **AVG** (the typical value), and **MIN/MAX** (smallest and largest). They all do the same magic trick — take a whole

column, hand you back a single number.

The sneaky part? NULL is not zero. Averaging 10, 20, 30, and a blank gives you 20, not 15 — because SQL just pretends the blank isn't there. That one detail trips up almost every beginner, and we show you exactly how to handle it.

Clear examples, no jargon, beginner friendly. Come learn it with us! 📌

Reddit Summary

A clean mental model for SQL's five aggregate functions (and the NULL trap that breaks reports)

Sharing a breakdown I put together for folks learning SQL. The functions are COUNT, SUM, AVG, MIN, MAX, and the useful framing is: *many rows in, one value out*.

A few things that are worth knowing early because they cause real, silent bugs:

- **COUNT(*) vs COUNT(column):** `COUNT(*)` counts rows; `COUNT(column)` counts only non-NULL values. If you have 100 customers but 20 have no phone on file, those return 100 and 80 respectively. Both correct, different questions.
- **NULL is not zero:** `AVG` skips NULLs entirely rather than treating them as 0. `10, 20, 30, NULL` averages to 20, not 15. If you want blanks as zero, wrap with `COALESCE(col, 0)` first.
- **MIN/MAX work on dates:** handy for finding a first or last order without sorting.
- **You can't select a raw column beside an aggregate** without `GROUP BY` — one total can't line up with many distinct names.
- **Empty table gotcha:** `SUM` over zero rows returns NULL, not 0.

General guidance: aggregate in the database rather than pulling every row into your app — it's one pass, close to the data, and returns a tiny result. App-side only wins when the logic outgrows SQL's math.

Happy to answer questions on any of these in the comments.

GITHUB README

```
# SQL Aggregate Functions – COUNT, SUM, AVG, MIN, MAX
```

Lesson 7 of the **Fun with Learning Technology** SQL series. Learn how to collapse an entire table into a single, meaningful answer — and how to avoid the NULL traps that quietly corrupt results.

Problem

You have a column full of rows and you want **one** number: a total, an average, a count, or an extreme. That's aggregation – many values in, one value out.

Intuition

Say the question out loud and the function picks itself:

Question	Function
How many?	<code>`COUNT`</code>
How much altogether?	<code>`SUM`</code>
What's typical?	<code>`AVG`</code>
Smallest / largest?	<code>`MIN` / `MAX`</code>

Every one of these **ignores NULL** – except ``COUNT(*)``, which counts every row unconditionally.

Approach

- ``COUNT(*)`` counts rows; ``COUNT(col)`` counts non-NULL values.
- ``SUM`` and ``AVG`` are numeric only.
- ``AVG`` skips NULLs – it does **not** treat them as zero. Use ``COALESCE(col, 0)`` if you want blanks counted as zero.
- ``MIN`/`MAX`` work on numbers, dates, and text.
- You cannot select a raw column beside an aggregate without ``GROUP BY``.
- Prefer aggregating in SQL over pulling all rows into application code.

Python Solution

```
```python
import sqlite3

def summarize(conn: sqlite3.Connection) -> dict:
 """Run the five core aggregates in a single database round trip."""
 row = conn.execute(
 """
 SELECT
 COUNT(*) AS total_rows,
 COUNT(phone) AS phones_on_file, -- skips NULLs
 SUM(amount) AS revenue,
 AVG(amount) AS typical_order, -- NULLs ignored
 MIN(amount) AS smallest,
 MAX(amount) AS largest,
 MIN(ordered_on) AS first_order, -- works on dates
 MAX(ordered_on) AS last_order,
 AVG(COALESCE(amount, 0)) AS avg_blanks_as_zero
 FROM orders
 """
)
```

```
.fetchone()
keys = [
 "total_rows", "phones_on_file", "revenue", "typical_order",
 "smallest", "largest", "first_order", "last_order",
 "avg_blanks_as_zero",
]
return dict(zip(keys, row))
```

Run `python solution.py` to see the report and the built-in assert checks.

## Complexity

---

### Metric Value

### Why

Time  $O(n)$  One linear pass over the column

Space  $O(1)$  Only running state (total/count/min) is kept

## Video

---

 Watch the lesson: {LINK}

## Article

---

 Full write-up with examples, dry run, edge cases, and common mistakes: `sql-aggregate-functions-count-sum-avg-min-max`

Part of the **Fun with Learning Technology** SQL series. Next lesson: `GROUP BY` — one total *per group*. ``

## Quick Review

---

**SQL: when you need a single summary value from many rows, what do you reach for?**

Aggregation functions (COUNT, SUM, AVG, MIN, MAX). Tell: the question is about the 'whole' — totals, averages, extremes — not individual rows.

**Time complexity of a single aggregate over N rows?**

$O(N)$  — the engine scans each row once to fold it into the running result. An index can cut it, but the base scan is linear.

**Space complexity of COUNT/SUM/AVG/MIN/MAX?**

$O(1)$  — one accumulator held while scanning. With GROUP BY it becomes  $O(g)$ , one accumulator per distinct group key.

**Common bug: how do NULLs affect COUNT and SUM?**

Aggregates skip NULLs — they aren't treated as zero. AVG divides by non-NULL count only, so NULLs quietly change the result versus what you'd expect.

**COUNT(\*) vs COUNT(col): what's the difference?**

COUNT(\*) counts all rows. COUNT(col) counts only rows where col is non-NULL. They diverge whenever the column has NULLs.

**Bug: SELECT name, COUNT(\*) FROM t; — why does this break?**

Mixing a raw column with an aggregate needs GROUP BY name. Without it, the row-level column has no single value to pair with the collapsed aggregate.

---

sql Lesson 7: Aggregation Functions — COUNT, SUM, AVG, MIN, MAX (Aggregation functions are mathematical operators designed to reduce large datasets into single, meaningful values based on specific calculations. These functions are essential for deriving business intelligence and summary statistics from raw transactional data. You reach for them whenever you need to answer questions about the 'whole' rather than the 'parts,' such as calculating totals, averages, or identifying extremes.) — free Python cheat sheet